MiniMu: A Language for Understanding Duality

DING FENG, National University of Singapore, Singapore, ACM membership number: 4708481 (Undergraduate)
KYRIEL MORTEL ABAD, National University of Singapore, Singapore, ACM membership number: 4353394 (Undergraduate)
MICHAEL D. ADAMS*, National University of Singapore, Singapore

In recent years, dual languages—grounded in treating continuations as first-class citizens—have emerged as a promising programming paradigm. Notable among these is Duo [4], an implementation of the $\lambda_{\mu\tilde{\mu}}$-calculus [2]. However, Duo is a large language that can make it difficult to see the essence of dual languages.

In this work, we present *MiniMu*, a language based on the μ and $\tilde{\mu}$ constructs without the λ construct. *MiniMu* conceptualizes computation as flows of data, emphasizing the fundamental duality between producers and consumers—or more technically, between expressions and continuations—through its core construct: the *command*. For data representation, which encompasses both conventional data structures and first-class continuations, we employ the notion of *constructors* and μ expressions. The resulting paradigm exposes the control flow in a unified, easy-to-use and concise way.

ACM Reference Format:
Ding Feng, Kyriel Mortel Abad, and Michael D. Adams. 2025. MiniMu: A Language for Understanding Duality. 1, 1 (April 2025), 3 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Traditional functional programming languages are based on the λ-calculus, where program execution is driven by the application of functions and higher-level concepts are encoded in terms of such functions. These languages also treat control-flow operators (e.g. continuations or *shift/reset* [1]) as second-class citizens. This makes continuations and things like continuation-passing style (CPS) [3] obscure and difficult for new programmers to understand.

In this work, we introduce a new programming language called *MiniMu* that departs from traditional λ-calculus-based languages by building upon the $\mu\tilde{\mu}$-calculus. Grounded in sequent calculus, *MiniMu* emphasizes the idea that "the structure of the program follows the structure of data" [5] by implementing an alternative control flow model using μ and $\tilde{\mu}$ terms. By making all aspects of control flow explicit, this language enables direct manipulation of continuations.

*Research Advisor

Authors' Contact Information: Ding Feng, dingfeng@u.nus.edu, National University of Singapore, Singapore , ACM membership number: 4708481 (Undergraduate); Kyriel Mortel Abad, kyriel@u.nus.edu, National University of Singapore, Singapore , ACM membership number: 4353394 (Undergraduate); Michael D. Adams, National University of Singapore, Singapore.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2025/4-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 Literature Review

This work focuses on the duality of computation that is found in different aspects.

(1) **Duality of computation.** Curien and Herbelin [2] explore symmetries inherent in programming languages and computation, in particular the duality between call-by-name (CBN) and call-by-value (CBV) evaluation strategies, as well as program and context duality. They argue that while the traditional Curry-Howard correspondence connects programs in the λ-calculus to proofs via natural deduction, sequent calculus offers a better framework for revealing computational dualities. They developed $\lambda\mu\tilde{\mu}$–calculus as "sequent calculus version" of the λ-calculus. Moving beyond natural deduction by extending λ-calculus with control abstractions.

(2) **Duality between call-by-name and call-by-value.** Wadler [6] argues for a fundamental duality between call-by-value (CBV) and call-by-name (CBN) evaluation strategies. This duality is analogous to De Morgan's duality in classical logic. Implication, analogous to a computational step, is hence defined differently for the two paradigms, similar to De Morgan's duality: $A \supset B \equiv \neg(A \wedge \neg B)$ for CBV and $A \supset B \equiv \neg A \vee B$ for CBN. It builds connections between computation, logic, and continuations.

(3) **Merge sequent calculus into functional programming.** Ostermann et al. [5] investigates the design space of programming languages based on logical rules, moving beyond the traditional dichotomy between *natural deduction* and *sequent calculus*. It proposes a unified framework incorporating four types of rules (i.e., introduction and elimination, both left and right) and argues that this provides greater flexibility for programmers. The framework follows the idea of "the structure of the program follows the structure of the data". It positions *commands* as the basic structure of the calculus and provides the translation between this new framework and the CPS-based λ-calculus. In essence, this work synthesizes ideas from natural deduction, sequent calculus, and duality principles to propose a richer foundational calculus for programming that provides programmers with more control over program structure.

(4) **Dual programming languages.** The *Duo* programming language [4] is an experimental language inspired by the ideas presented in [5]. It is *command-driven* and incorporates definitions of *data* and *codata*, which are used to represent finite and infinite structures such as *constructors* and *observers* respectively. These concepts reflect the duality between expressions and continuations and between CBV and CBN evaluation strategies. The keywords *case* and *cocase* are used to perform sequential deduction, aligning with its foundational emphasis on the flow of data.

3 Contribution

Building upon the theoretical foundation of the $\mu\tilde{\mu}$-calculus, this research proposes a new programming language, *MiniMu*, which emphasizes duality in computation, with the resulting explicit control-flow. *MiniMu* relies solely on $\mu\tilde{\mu}$-abstraction, avoiding the broader $\lambda\mu\tilde{\mu}$ system used in languages like Duo[4]. It provides cleaner reasoning, a more uniform meta-theory and the concepts of code-as-control and control-as-code, compared to traditional ways of making control explicit. It remains small in scale and clear in its foundational concepts, offering an accessible, minimal and focused framework for understanding continuation-related concepts or dualities in computing.

Current work focuses not only on implementing the language, but also on exploring it by developing example programs—drawn from traditional functional programming—within this new paradigm, with the goal of discovering its natural idioms.

4 Design Overview

The interpreter of *MiniMu* is implemented in Haskell. A program in *MiniMu* is defined as a sequence of declarations, where each declaration represents a binding between a *(co)expression* and a corresponding *(co)variable*. The notion of *(co)expression* is modeled as a sum type, comprising three fundamental constructs: *(co)variables*, *(co)constructors*, and *(co)mu* operators.

Among these, the *(co)mu* operators play a central role in the system, serving as the computational core of the language, mapping matched *(co)expressions* to *commands*. A *command* is the core of the dynamics of the language. It features a dual pair of an expression and a co-expression (i.e., continuation), which as a whole represents current focus point in the evaluation process. The explicit handling of expression and continuation structures facilitates precise control over the flow of computation.

5 Conclusion

Through the development of *MiniMu*'s interpreter and sample programs for it, the research demonstrates how the language's dual continuation-expression model offers advantages over traditional functional programming. This model brings greater clarity to programmers, making control flow explicit and visible, rather than hidden and relegated to second-class-citizen status. This enhances language expressiveness, allowing the integration of imperative patterns directly into a functional core—something that has traditionally been difficult to achieve well. We gain more abstraction power, allowing the construction of custom control structures without needing built-in operators. Furthermore, its semantic elegance means that fundamental language features (e.g., functions, pattern-matching, and loops) can be built from the same few basic primitives. This structural innovation inherently simplifies the often-required approach of using continuation-passing-style (CPS) translation in traditional functional programming.

References

[1] Dariusz Biernacki, Olivier Danvy, and Chung chieh Shan. 2006. On the static and dynamic extents of delimited continuations. *Science of Computer Programming* 60, 3 (2006), 274–297. doi:10.1016/j.scico.2006.01.002

[2] Pierre-Louis Curien and Hugo Herbelin. 2000. The duality of computation. *SIGPLAN Not.* 35, 9 (Sept. 2000), 233–243. doi:10.1145/357766.351262

[3] Oliver Danvy and Andrzex Filinski. 1992. Representing Control: a Study of the CPS Transformation. *Mathematical Structures in Computer Science* 2, 4 (1992), 361–391. doi:10.1017/S0960129500001535

[4] Duo. 2023. https://duo-lang.github.io/documentation/preface.html. Accessed: 2025-03-30.

[5] Klaus Ostermann, David Binder, Ingo Skupin, Tim Süberkrüb, and Paul Downen. 2022. Introduction and elimination, left and right. *Proc. ACM Program. Lang.* 6, ICFP, Article 106 (Aug. 2022), 28 pages. doi:10.1145/3547637

[6] Philip Wadler. 2003. Call-by-value is dual to call-by-name. *SIGPLAN Not.* 38, 9 (Aug. 2003), 189–201. doi:10.1145/944746.944723